

ASSIGNMENT 13

Hemavathi N

2303A51965

Batch 24

AIAC

Task Description #1 (Refactoring – Removing Code Duplication)

- Task: Use AI to refactor a given Python script that contains multiple repeated code blocks.

- Instructions:

- o Prompt AI to identify duplicate logic and replace it with functions or classes.

- o Ensure the refactored code maintains the same output.

- o Add docstrings to all functions.

- Sample Legacy Code:

Legacy script with repeated logic

```
print("Area of Rectangle:", 5 * 10)
print("Perimeter of Rectangle:", 2 * (5 + 10))
print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))
print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15))
```

- Expected Output:

- o Refactored code with a reusable function and no duplication.

- o Well documented code

```
C:\Users\ADMIN\OneDrive\Desktop\AIAC\Q1.py calculate_rectangle_properties
1 # Legacy script with repeated logic
2 print("Area of Rectangle:", 5 * 10)
3 print("Perimeter of Rectangle:", 2 * (5 + 10))
4 print("Area of Rectangle:", 7 * 12)
5 print("Perimeter of Rectangle:", 2 * (7 + 12))
6 print("Area of Rectangle:", 10 * 15)
7 print("Perimeter of Rectangle:", 2 * (10 + 15))
8 #Refactored code with a reusable function and no duplication.
9 #Well documented code
10 def calculate_rectangle_properties(length, width):
11     """
12     Calculate and print the area and perimeter of a rectangle.
13
14     Parameters:
15     length (float): The length of the rectangle.
16     width (float): The width of the rectangle.
17     """
18     area = length * width
19     perimeter = 2 * (length + width)
20
21     print(f"Area of Rectangle: {area}")
22     print(f"Perimeter of Rectangle: {perimeter}")
23 # Example usage
24 calculate_rectangle_properties(5, 10)
25 calculate_rectangle_properties(7, 12)
```

OUTPUT :-

```
PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.exe" c:/Users/ADMIN/OneDrive/Desktop/AIAC/Q1.py
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 30
```

Task Description #2 (Refactoring – Extracting Reusable Functions)

- Task: Use AI to refactor a legacy script where multiple calculations are embedded directly inside the main code block.

- Instructions:

- o Identify repeated or related logic and extract it into reusable functions.

- o Ensure the refactored code is modular, easy to read, and documented with docstrings.

- Sample Legacy Code:

Legacy script with inline repeated logic

```
price = 250
```

```
tax = price * 0.18
```

```
total = price + tax
```

```
print("Total Price:", total)
```

```
price = 500
```

```
tax = price * 0.18
```

```
total = price + tax
```

```
print("Total Price:", total)
```

- Expected Output:

- o Code with a function `calculate_total(price)` that can be reused for multiple price inputs.

- o Well documented code

```

C:\Users\ADMIN> OneDrive\Desktop\AIAC> Q2.py> calculate_total
1 # Legacy script with inline repeated logic
2 price = 250
3 tax = price * 0.18
4 total = price + tax
5 print("Total Price:", total)
6 price = 500
7 tax = price * 0.18
8 total = price + tax
9 print("Total Price:", total)
10 #Code with a function calculate_total(price) that can be reused for multiple price inputs.
11 #Well documented code
12 def calculate_total(price):
13     """
14     Calculate and print the total price including tax.
15
16     Parameters:
17     price (float): The base price of the item.
18     """
19     tax = price * 0.18
20     total = price + tax
21     print("Total Price:", total)
22 # Example usage
23 calculate_total(250)
24 calculate_total(500)

```

OUTPUT :-

```

PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.exe" c:/Users/ADMIN/OneDrive/Desktop/AIAC/Q2
Total Price: 295.0
Total Price: 590.0
Total Price: 295.0
Total Price: 590.0

```

Task Description #3: Refactoring Using Classes and Methods (Eliminating Redundant Conditional Logic)

Refactor a Python script that contains repeated if–elif–else grading logic by implementing a structured, object-oriented solution using a class and a method.

Problem Statement

The given script contains duplicated conditional statements used to assign grades based on student marks. This redundancy violates clean code principles and reduces maintainability.

You are required to refactor the script using a class-based design to improve modularity, reusability, and readability while preserving the original grading logic.

Mandatory Implementation Requirements

1. Class Name: GradeCalculator
2. Method Name: calculate_grade(self, marks)
3. The method must:
 - o Accept marks as a parameter.
 - o Return the corresponding grade as a string.
 - o The grading logic must strictly follow the conditions below:
 - Marks ≥ 90 and $\leq 100 \rightarrow$ "Grade A"
 - Marks $\geq 80 \rightarrow$ "Grade B"
 - Marks $\geq 70 \rightarrow$ "Grade C"

- Marks $\geq 40 \rightarrow$ "Grade D"

- Marks $\geq 0 \rightarrow$ "Fail"

Note: Assume marks are within the valid range of 0 to 100.

4. Include proper docstrings for:

- o The class

- o The method (with parameter and return descriptions)

5. The method must be reusable and called multiple times without rewriting conditional logic.

- Given code:

```
marks = 85
```

```
if marks >= 90:
```

```
    print("Grade A")
```

```
elif marks >= 75:
```

```
    print("Grade B")
```

```
else:
```

```
    print("Grade C")
```

```
marks = 72
```

```
if marks >= 90:
```

```
    print("Grade A")
```

```
elif marks >= 75:
```

```
    print("Grade B")
```

```
else:
```

```
    print("Grade C")
```

Expected Output:

- Define a class named GradeCalculator.
- Implement a method calculate_grade(self, marks) inside the class.
- Create an object of the class.
- Call the method for different student marks.
- Print the returned grade values.

```
C:\> Users\ADMIN\OneDrive\Desktop\AIAC\Q3.py > ...
1  marks = 85
2  if marks >= 90:
3      print("Grade A")
4  elif marks >= 75:
5      print("Grade B")
6  else:
7      print("Grade C")
8  marks = 72
9  if marks >= 90:
10     print("Grade A")
11 elif marks >= 75:
12     print("Grade B")
13 else:
14     print("Grade C")
15
16 class GradeCalculator:
17     def calculate_grade(self, marks):
18         """
19         Calculate and return the grade based on the marks.
20
21         Parameters:
22         marks (float): The marks obtained by the student
23         """
```

OUTPUT :-

```
PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.exe" c:/Users/ADMIN/OneDrive/Desktop/AI
Grade B
Grade C
Grade B
Grade C
```

Task Description #4 (Refactoring – Converting Procedural Code to Functions)

- Task: Use AI to refactor procedural input–processing logic into functions.

Instructions:

- o Identify input, processing, and output sections.
- o Convert each into a separate function.
- o Improve code readability without changing behavior.

- Sample Legacy Code:

```
num = int(input("Enter number: "))
```

```
square = num * num
```

```
print("Square:", square)
```

- Expected Output:

- o Modular code using functions like `get_input()`, `calculate_square()`, and `display_result()`.

```
C:\> Users > ADMIN > OneDrive > Desktop > AIAC > Q4.py > ...
1  #Sample Legacy Code:
2  num = int(input("Enter number: "))
3  square = num * num
4  print("Square:", square)
5  #Refactored code with functions like get_input(), calculate_square(), and display_result()
6  def get_input():
7      """
8      Prompt the user to enter a number and return it as an integer.
9
10     Returns:
11     int: The number entered by the user.
12     """
13     return int(input("Enter number: "))
14  def calculate_square(num):
15      """
16      Calculate the square of a number.
17      Parameters:
18      num (int): The number to be squared.
19      Returns:
20      int: The square of the input number.
21      """
22     return num * num
23  def display_result(square):
24      """
25     Display the square of the number.
```

OUTPUT :-

```
PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.exe" c:/Users/ADMIN/OneD
Enter number: 2
Square: 4
Enter number: 4
Square: 16
```

Task 5 (Refactoring Procedural Code into OOP Design)

- Task: Use AI to refactor procedural code into a class-based design.

Focus Areas:

- o Object-Oriented principles
- o Encapsulation

Legacy Code:

```
salary = 50000
tax = salary * 0.2
net = salary - tax
print(net)
```

Expected Outcome:

- o A class like EmployeeSalaryCalculator with methods and attributes.

```
C:\> Users > ADMIN > OneDrive > Desktop > AIAC > Q5 13.py > ...

1  #Legacy Code:
2  salary = 50000
3  tax = salary * 0.2
4  net = salary - tax
5  print(net)
6  #refactored code with a class like EmployeeSalaryCalculator with methods and attributes.
7  class EmployeeSalaryCalculator:
8      def __init__(self, salary):
9          """
10             Initialize the EmployeeSalaryCalculator with the given salary.
11
12             Parameters:
13             salary (float): The gross salary of the employee.
14             """
15             self.salary = salary
16
17     def calculate_tax(self):
18         """
19             Calculate the tax based on the salary.
20
21             Returns:
22             float: The calculated tax amount.
23             """
24         return self.salary * 0.2
```

OUTPUT :-

```
PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.exe" "c:/Users/ADMIN/OneDrive/Desktop/AIAC/Q5 13.py"
40000.0
40000.0
```

Task 6 (Optimizing Search Logic)

- Task: Refactor inefficient linear searches using appropriate data structures.

• Focus Areas:

- o Time complexity
- o Data structure choice

Legacy Code:

```
users = ["admin", "guest", "editor", "viewer"]
name = input("Enter username: ")
found = False
for u in users:
```

```
if u == name:
found = True
print("Access Granted" if found else "Access Denied")
```

Expected Outcome:

o Use of sets or dictionaries with complexity justification

```
C: > Users > ADMIN > OneDrive > Desktop > AIAC > Q6.py > ...
1 #Refactor inefficient linear search using data structures like sets and reduce time complexity
2
3 users = {"admin", "guest", "editor", "viewer"}
4 name = input("Enter username: ")
5 found = name in users
6 print("Access Granted" if found else "Access Denied")
```

OUTPUT :-

```
PS C:\Users\ADMIN\OneDrive\Desktop\AIAC> & "C:/Program Files/Python313/Python.exe" Q6.py
Enter username: admin
Access Granted
```

Task 7 – Refactoring the Library Management System

Problem Statement

You are provided with a poorly structured Library Management script that:

- Contains repeated conditional logic
- Does not use reusable functions
- Lacks documentation
- Uses print-based procedural execution
- Does not follow modular programming principles

Your task is to refactor the code into a proper format

1. Create a module library.py with functions:

- o add_book(title, author, isbn)
- o remove_book(isbn)
- o search_book(isbn)

2. Insert triple quotes under each function and let Copilot complete the docstrings.

3. Generate documentation in the terminal.

4. Export the documentation in HTML format.

5. Open the file in a browser.


```
C:\Users\ADMIN\OneDrive\Desktop\AIAC> library.py > ...

39 isbn = "101"
40 if isbn in library_db:
41     print("Book Found:", library_db[isbn])
42 else:
43     print("Book not found.")
44 # refactored code with Create a module library.py with functions:
45 # add_book(title, author, isbn)
46 # remove_book(isbn)
47 # search_book(isbn)
48 # 2.Insert triple quotes under each function and let Copilot complete the docstrings.
49 # 3.Generate documentation in the terminal.
50 # 4.Export the documentation in HTML format.
51 # 5.Open the file in a browser.
52 def add_book(title, author, isbn):
53     """
54     Add a book to the library database.
55
56     Parameters:
57     title (str): The title of the book.
58     author (str): The author of the book.
59     isbn (str): The ISBN number of the book.
60
61     Returns:
62     str: A message indicating whether the book was added successfully or if it already exists.
63     """
64     if isbn not in library_db:
65         library_db[isbn] = {"title": title, "author": author}
66         return "Book added successfully."
67     else:
```

OUTPUT :-

```
> TERMINAL Python + x
PS C:\Users\ADMIN\OneDrive\Desktop\AIAC> python -m pydoc -m library
Book added successfully.
Book Found: {'title': 'Python Basics', 'author': 'John Doe'}
Book removed successfully.
Book not found.
Book added successfully.
Book already exists.
Book Found: {'title': 'Python Basics', 'author': 'John Doe'}
Book removed successfully.
Book not found.
wrote library.html
PS C:\Users\ADMIN\OneDrive\Desktop\AIAC> start library.html
PS C:\Users\ADMIN\OneDrive\Desktop\AIAC> |
```

[index](#)
library <c:/users/admin/onedrive/desktop/aiaclibrary.py>

Library Management System (Unstructured Version)
This code needs refactoring into a proper module with documentation.

Functions

add_book(title, author, isbn)
Add a book to the library database.

Parameters:
title (str): The title of the book.
author (str): The author of the book.
isbn (str): The ISBN number of the book.

Returns:
str: A message indicating whether the book was added successfully or if it already exists.

remove_book(isbn)
Remove a book from the library database.

Parameters:
isbn (str): The ISBN number of the book to be removed.

Returns:
str: A message indicating whether the book was removed successfully or if it was not found.

search_book(isbn)
Search for a book in the library database.

Task 8– Fibonacci Generator

Write a program to generate Fibonacci series up to n.

The initial code has:

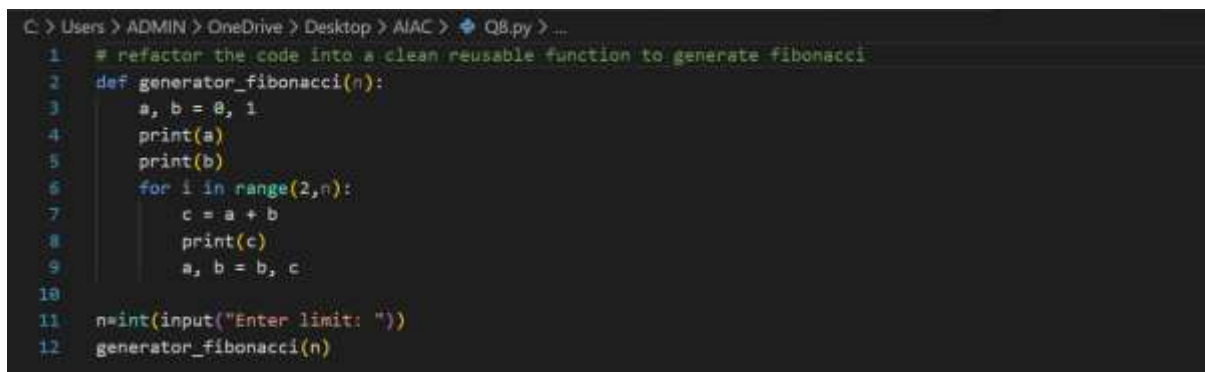
- Global variables.
- Inefficient loop.
- No functions or modularity.

Task for Students:

- Refactor into a clean reusable function (generate_fibonacci).
- Add docstrings and test cases.
- Compare AI-refactored vs original.

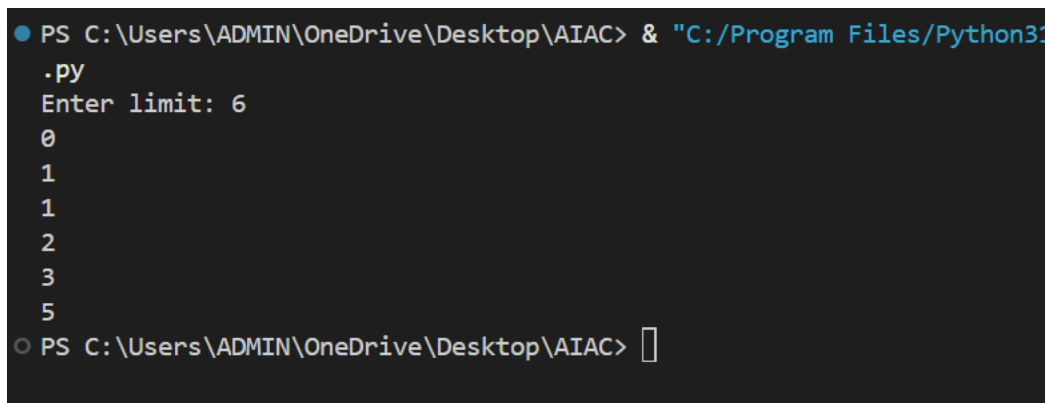
Bad Code Version:

```
# fibonacci bad version
n=int(input("Enter limit: "))
a=0
b=1
print(a)
print(b)
for i in range(2,n):
    c=a+b
    print(c)
    a=b
    b=c
```



```
C:\Users\ADMIN> OneDrive\Desktop\AIAC> Q8.py > ...
1  # refactor the code into a clean reusable function to generate fibonacci
2  def generator_fibonacci(n):
3      a, b = 0, 1
4      print(a)
5      print(b)
6      for i in range(2,n):
7          c = a + b
8          print(c)
9          a, b = b, c
10
11  n=int(input("Enter limit: "))
12  generator_fibonacci(n)
```

OUTPUT :-



```
PS C:\Users\ADMIN\OneDrive\Desktop\AIAC> & "C:/Program Files/Python3...
.py
Enter limit: 6
0
1
1
2
3
5
PS C:\Users\ADMIN\OneDrive\Desktop\AIAC> 
```

Task 9 – Twin Primes Checker

Twin primes are pairs of primes that differ by 2 (e.g., 11 and 13, 17 and 19).

The initial code has:

- Inefficient prime checking.
- No functions.
- Hardcoded inputs.

Task for Students:

- Refactor into `is_prime(n)` and `is_twin_prime(p1, p2)`.
- Add docstrings and optimize.
- Generate a list of twin primes in a given range using AI.

Bad Code Version:

```
# twin primes bad version
```

```
a=11
```

```
b=13
```

```
fa=0
```

```
for i in range(2,a):
```

```
    if a%i==0:
```

```
        fa=1
```

```
fb=0
```

```
for i in range(2,b):
```

```
    if b%i==0:
```

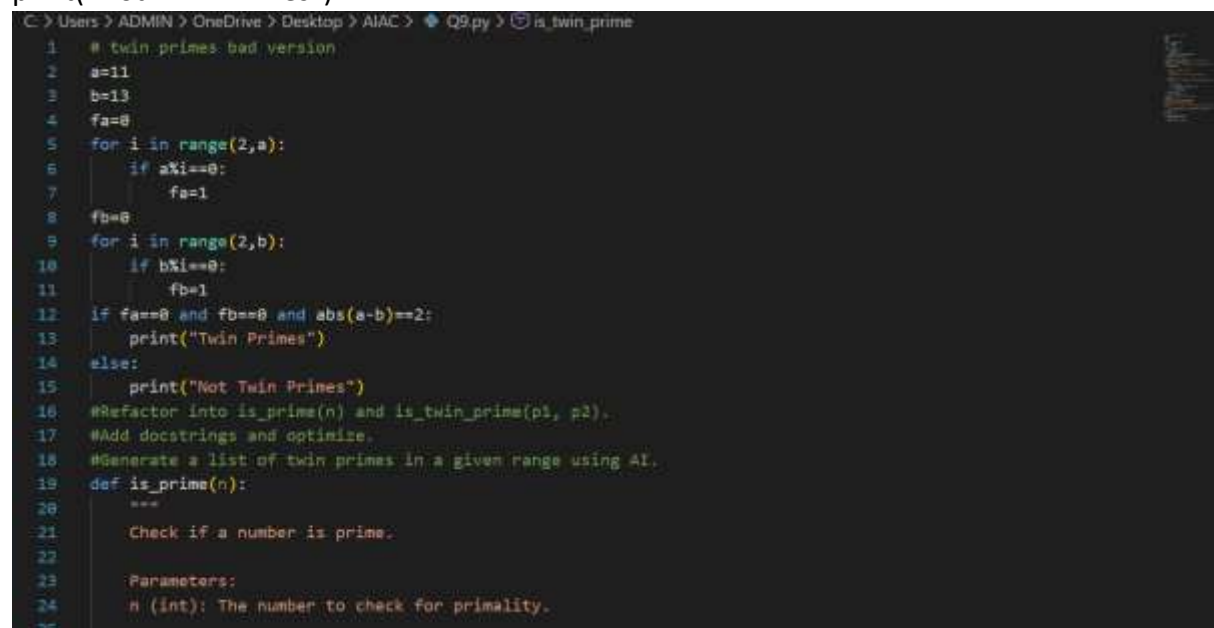
```
        fb=1
```

```
if fa==0 and fb==0 and abs(a-b)==2:
```

```
    print("Twin Primes")
```

```
else:
```

```
    print("Not Twin Primes")
```



```
C:\Users\ADMIN> OneDrive\Desktop> AIAC > Q9.py > a_twin_prime
1 # twin primes bad version
2 a=11
3 b=13
4 fa=0
5 for i in range(2,a):
6     if a%i==0:
7         fa=1
8 fb=0
9 for i in range(2,b):
10     if b%i==0:
11         fb=1
12 if fa==0 and fb==0 and abs(a-b)==2:
13     print("Twin Primes")
14 else:
15     print("Not Twin Primes")
16 #Refactor into is_prime(n) and is_twin_prime(p1, p2).
17 #Add docstrings and optimize.
18 #Generate a list of twin primes in a given range using AI.
19 def is_prime(n):
20     """
21     Check if a number is prime.
22
23     Parameters:
24     n (int): The number to check for primality.
25 """
```

OUTPUT :-

```
PS C:\Users\ADMIN\OneDrive\Desktop\AIAC> & "C:/Program Files/Python313/python.exe" c:/Users/ADMIN/.py
Twin Primes
Twin Primes
```

Task 10 – Refactoring the Chinese Zodiac Program

Objective

Refactor the given poorly structured Python script into a clean, modular, and reusable implementation.

The current program reads a year from the user and prints the corresponding Chinese Zodiac sign. However, the implementation contains repetitive conditional logic, lacks modular design, and does not follow clean coding principles.

Your task is to refactor the code to improve readability, maintainability, and structure.

```
C > Users > ADMIN > OneDrive > Desktop > AIAC > Q10.py > get_zodiac
1 def get_zodiac(year):
2     """
3     Determine the Chinese Zodiac sign for a given year.
4     Parameters:
5     year (int): The year for which the zodiac sign is required.
6     Returns:
7     str: The corresponding Chinese Zodiac sign.
8     """
9     zodiac_cycle = [
10         "Monkey", # 0
11         "Rooster", # 1
12         "Dog", # 2
13         "Pig", # 3
14         "Rat", # 4
15         "Ox", # 5
16         "Tiger", # 6
17         "Rabbit", # 7
18         "Dragon", # 8
19         "Snake", # 9
20         "Horse", # 10
21         "Goat", # 11
```

OUTPUT :-

```
Enter a year: 2024
Chinese Zodiac Sign: Dragon
PS C:\Users\ADMIN\OneDrive\Desktop\AIAC>
```

Task 11 – Refactoring the Harshad (Niven) Number Checker

Refactor the given poorly structured Python script into a clean, modular, and reusable implementation.

A Harshad (Niven) number is a number that is divisible by the sum of its digits.

For example:

- $18 \rightarrow 1 + 8 = 9 \rightarrow 18 \div 9 = 2$ (Harshad Number)
- $19 \rightarrow 1 + 9 = 10 \rightarrow 19 \div 10 \neq \text{integer}$ (Not Harshad)

Problem Statement

The current implementation:

- Mixes logic and input handling
- Uses redundant variables

- Does not use reusable functions properly
- Returns print statements instead of boolean values
- Lacks documentation

You must refactor the code to follow clean coding principles.

```
C:\Users\ADMIN> OneDrive\Desktop\AIAC> Q11.py > ...
1  # Refactor the code by adding a function is_harshad(number) to improve readability and reusability
2  def is_harshad(number):
3      """
4      Check if a number is a Harshad number.
5      Parameters:
6      number (int): The number to check.
7      Returns:
8      bool: True if the number is a Harshad number, False otherwise.
9      """
10     if number < 0:
11         return False
12     temp = number
13     sum_digits = 0
14     while temp > 0:
15         digit = temp % 10
16         sum_digits = sum_digits + digit
17         temp = temp // 10
18     if sum_digits != 0:
19         return number % sum_digits == 0
20     else:
21         return False
22 # Example usage:
23 num = int(input("Enter a number: "))
24 if is_harshad(num):
```

OUTPUT :-

```
PS C:\Users\ADMIN\OneDrive\Desktop\AIAC> "C:/Program Files/Python313/python.exe" c:/Users/ADMIN/OneDrive/Desktop/AIAC/Q11.py
Enter a number: 18
18 is a Harshad number.
PS C:\Users\ADMIN\OneDrive\Desktop\AIAC>
```

Task 12 – Refactoring the Factorial Trailing Zeros Program

Refactor the given poorly structured Python script into a clean, modular, and efficient implementation.

The program calculates the number of trailing zeros in $n!$ (factorial of n).

Problem Statement

The current implementation:

- Calculates the full factorial (inefficient for large n)
- Mixes input handling with business logic
- Uses print statements instead of return values
- Lacks modular structure and documentation

You must refactor the code to improve efficiency, readability, and maintainability.

```

C:\Users\ADMIN> OneDrive\Desktop\AIAC> Q12.py > ...
1  #Refactor the code to improve efficiency, readability, and maintainability.
2  def count_trailing_zeros(n):
3      """
4      Count the number of trailing zeros in n! (n factorial).
5      Parameters:
6      n (int): A non-negative integer.
7      Returns:
8      int: The number of trailing zeros in n!.
9      """
10     if n < 0:
11         return 0
12     count = 0
13     power_of_5 = 5
14     while power_of_5 <= n:
15         count += n // power_of_5
16         power_of_5 *= 5
17     return count
18 def main():
19     #Handle user input and display the number of trailing zeros in n!.
20     try:
21         n = int(input("Enter a non-negative integer: "))
22         if n < 0:
23             print("Please enter a non-negative integer.")

```

OUTPUT :-

```

PS C:\Users\ADMIN\OneDrive\Desktop\AIAC> & "C:/Program Files/Python39/Python.exe" C:\Users\ADMIN\OneDrive\Desktop\AIAC\Q12.py
Enter a non-negative integer: 12
The number of trailing zeros in 12! is: 2
PS C:\Users\ADMIN\OneDrive\Desktop\AIAC>

```

Task 13 (Collatz Sequence Generator – Test Case Design)

- Function: Generate Collatz sequence until reaching 1.
- Test Cases to Design:
- Normal: $6 \rightarrow [6, 3, 10, 5, 16, 8, 4, 2, 1]$
- Edge: $1 \rightarrow [1]$
- Negative: -5
- Large: 27 (well-known long sequence)
- Requirement: Validate correctness with pytest.

```

C:\Users\ADMIN> OneDrive\Desktop\AIAC> collatz.py > test_collatz_negative
1  import pytest
2  def collatz_sequence(n: int) -> list[int]:
3      if not isinstance(n, int) or n <= 0:
4          raise ValueError("Input must be a positive integer")
5      sequence = [n]
6      while n != 1:
7          if n % 2 == 0:
8              n = n // 2
9          else:
10             n = 3 * n + 1
11             sequence.append(n)
12     return sequence
13 def test_collatz_normal():
14     assert collatz_sequence(6) == [6, 3, 10, 5, 16, 8, 4, 2, 1]
15 def test_collatz_edge():
16     assert collatz_sequence(1) == [1]
17 def test_collatz_negative():
18     with pytest.raises(ValueError):
19         collatz_sequence(-5)
20 def test_collatz_large():
21     result = collatz_sequence(27)
22     assert result[0] == 27
23     assert result[-1] == 1
24     assert len(result) == 112

```

OUTPUT :-

```
PS C:\Users\ADMIN\OneDrive\Desktop\AIAC> & "C:/Program Files/Python313/python.exe" -m pytest collatz.py
===== test session starts =====
platform win32 -- Python 3.13.7, pytest-8.4.2, pluggy-1.6.0
rootdir: C:\Users\ADMIN\OneDrive\Desktop\AIAC
collected 4 items

collatz.py .... [100%]

===== 4 passed in 0.05s =====
```

Task 14 (Lucas Number Sequence – Test Case Design)

- Function: Generate Lucas sequence up to n terms.

(Starts with 2,1, then $F_n = F_{n-1} + F_{n-2}$)

- Test Cases to Design:

- Normal: 5 $\rightarrow$ [2, 1, 3, 4, 7]

- Edge: 1 $\rightarrow$ [2]

- Negative: -5 $\rightarrow$ Error

- Large: 10 (last element = 76).

- Requirement: Validate correctness with pytest.

```
C:\Users\ADMIN\OneDrive\Desktop\AIAC> test_lucas.py > test_lucas_large
1 import pytest
2 def lucas_sequence(n: int) -> list[int]:
3     if not isinstance(n, int) or n <= 0:
4         raise ValueError("Input must be a positive integer")
5     if n == 1:
6         return [2]
7     if n == 2:
8         return [2, 1]
9     sequence = [2, 1]
10    for _ in range(2, n):
11        sequence.append(sequence[-1] + sequence[-2])
12    return sequence
13 def test_lucas_normal():
14     assert lucas_sequence(5) == [2, 1, 3, 4, 7]
15 def test_lucas_edge():
16     assert lucas_sequence(1) == [2]
17 def test_lucas_negative():
18     with pytest.raises(ValueError):
19         lucas_sequence(-5)
20 def test_lucas_large():
21     result = lucas_sequence(10)
22     assert len(result) == 10
23     assert result[-1] == 76
```

OUTPUT :-

```
PS C:\Users\ADMIN\OneDrive\Desktop\AIAC> & "C:/Program Files/Python313/python.exe" -m pytest test_lucas.py
===== test session starts =====
platform win32 -- Python 3.13.7, pytest-8.4.2, pluggy-1.6.0
rootdir: C:\Users\ADMIN\OneDrive\Desktop\AIAC
collected 4 items

test_lucas.py .... [100%]

===== 4 passed in 0.04s =====
```

Task 15 (Vowel & Consonant Counter – Test Case Design)

- Function: Count vowels and consonants in string.

- Test Cases to Design:

- Normal: "hello" $\rightarrow$ (2,3)

- Edge: "" $\rightarrow$ (0,0)

- Only vowels: "aeiou" $\rightarrow$ (5,0)

Large: Long text

- Requirement: Validate correctness with pytest.

```
1 import pytest
2 def count_vowels_consonants(text: str) -> tuple[int, int]:
3     if not isinstance(text, str):
4         raise ValueError("Input must be a string")
5     vowels_set = set("aeiouAEIOU")
6     vowels = 0
7     consonants = 0
8     for ch in text:
9         if ch.isalpha():
10             if ch in vowels_set:
11                 vowels += 1
12             else:
13                 consonants += 1
14     return vowels, consonants
15 def test_normal():
16     assert count_vowels_consonants("hello") == (2, 3)
17 def test_edge_empty():
18     assert count_vowels_consonants("") == (0, 0)
19 def test_only_vowels():
20     assert count_vowels_consonants("aeiou") == (5, 0)
21 def test_large_text():
22     long_text = "This is a simple long text used for testing vowel and consonant counting." * 10
23     vowels, consonants = count_vowels_consonants(long_text)
24     assert vowels > 0
```

OUTPUT :-

```
PS C:\Users\ADMIN\OneDrive\Desktop\AIAC> & "C:/Program Files/Python313/python.exe" -m pytest test_vowel_counter.py
===== test session starts =====
platform win32 -- Python 3.13.7, pytest-8.4.2, pluggy-1.6.0
rootdir: C:\Users\ADMIN\OneDrive\Desktop\AIAC
collected 4 items

test_vowel_counter.py .... [100%]

===== 4 passed in 0.04s =====
```